

Reporte de práctica: Control de acceso con RFID RC522

Alumno	Jesús Antonio Rodríguez Peraza
Matrícula	231000105
Materia	Sistemas Programables
Práctica	7 — Control de acceso con RFID RC522 (bus SPI)
Plataforma	Arduino UNO R4 WiFi + lector RC522 + LED verde y rojo + matriz LED 12×8 integrada

Objetivo

Comunicar el Arduino con un lector RFID RC522 por medio del bus SPI, leer el número único (UID) de una tarjeta, mostrarlo en el Monitor Serie y usarlo para simular un control de acceso: la tarjeta autorizada enciende un LED verde durante 2 segundos y cualquier otra enciende el LED rojo. Como extra, la matriz LED de la placa muestra el resultado con texto.

Metodología

Bus SPI. El RC522 se conecta a los pines SPI fijos de la UNO: SCK (D13) marca el reloj, MOSI (D11) lleva los datos del Arduino al lector, MISO (D12) los trae de regreso, y SDA/SS (D10) selecciona al lector; RST (D9) lo reinicia. El módulo se alimenta con 3.3 V, porque a 5 V se daña. El programa llama a `SPI.begin()` y a `PCD_Init()` de la librería MFRC522.

Verificación de comunicación. Al arrancar se lee el registro `VersionReg` del lector. Un RC522 real responde 0x91 o 0x92; si llega 0x00 o 0xFF, nadie contestó por MISO, y el programa imprime "ERROR: no hay comunicacion con el RC522" y pide revisar el cableado SPI y los 3.3 V.

Lectura del UID. En cada vuelta de `loop()` se pregunta `PICC_IsNewCardPresent()` y `PICC_ReadCardSerial()`. El UID (normalmente 4 bytes) se imprime en hexadecimal, con dos dígitos por byte y separado por espacios: UID: 3A F2 1C 7B. Después se llama a `PICC_HaltA()` para que la misma tarjeta no se lea en ráfaga mientras sigue sobre el lector.

Control de acceso sin bloqueo. El UID leído se compara byte por byte con la lista AUTORIZADOS (el llavero 99 EB 7B 63 y la tarjeta AA 25 60 E4). Si coincide se imprime ACCESO CONCEDIDO y se enciende el LED verde (D7); si no, ACCESO DENEGADO y el LED rojo (D6). Cada LED es un `struct Led` con su pin, su estado y el instante en que se encendió; en cada `loop()` se revisa si ya pasaron 2000 ms para apagarlo. Como nunca hay un `delay()`, el programa sigue leyendo tarjetas mientras un LED está encendido.

Extra: texto en la matriz LED. La UNO R4 WiFi trae una matriz de 12×8 LEDs. Con las librerías `ArduinoGraphics` y `Arduino_LED_Matrix` se escribe ACEPTADO o RECHAZADO con la fuente de 4×6. La función de desplazamiento de la librería (`endText(ROLL_LEFT)`) bloquea el programa hasta que el texto termina de pasar, lo que rompería la regla de seguir leyendo tarjetas; por eso el desplazamiento se hace a mano: cada 60 ms (con `millis()`) el texto se redibuja una columna más a la izquierda, y cuando sale por completo la matriz se apaga.

Conexiones

RC522	Arduino UNO R4 WiFi	Función
SDA (SS)	D10	Selección del esclavo
SCK	D13	Reloj del bus SPI
MOSI	D11	Datos Arduino → lector
MISO	D12	Datos lector → Arduino
IRQ	sin conectar	No se usan interrupciones
GND	GND	Tierra común
RST	D9	Reinicio del lector
3.3V	3.3V	Alimentación (nunca 5 V)
LED verde	D7 → resistencia → LED → GND	Acceso concedido
LED rojo	D6 → resistencia → LED → GND	Acceso denegado

Diagrama de conexión

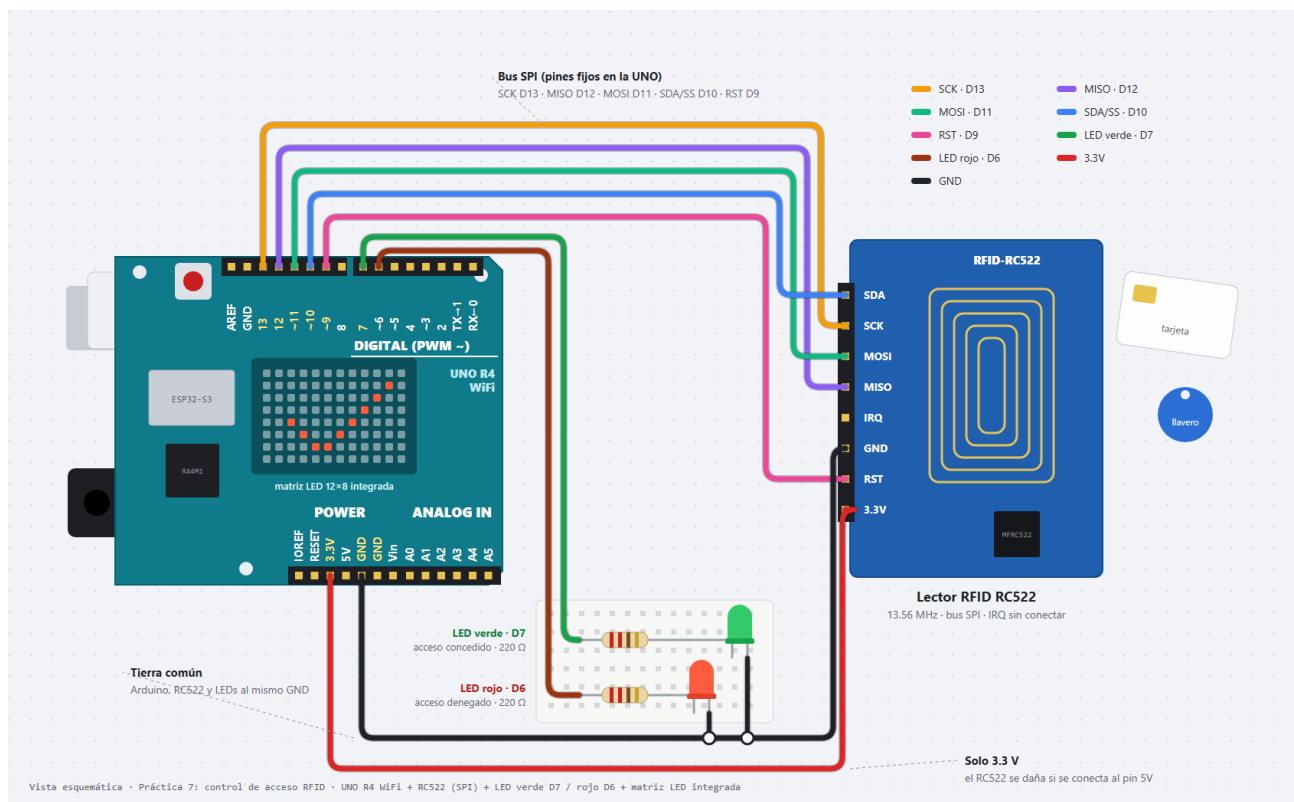


Diagrama de conexión: UNO R4 WiFi, lector RC522 por SPI, LED verde en D7, LED rojo en D6 y matriz LED integrada.

Pruebas

Estas son las pruebas que pide la consigna y lo que el programa está diseñado para mostrar en cada una:

Prueba	Resultado esperado
1. Acercar la tarjeta autorizada	UID + ACCESO CONCEDIDO; el LED verde enciende y se apaga solo a los 2 s; la matriz escribe ACEPTADO.
2. Acercar la tarjeta no autorizada	UID + ACCESO DENEGADO; el verde no enciende, se enciende el rojo 2 s; la matriz escribe RECHAZADO.
3. Con el LED encendido, acercar la no autorizada	Aparece ACCESO DENEGADO de inmediato y se enciende el rojo mientras el verde termina su conteo: el programa no estaba bloqueado.
4. Desconectar MISO (D12) y reiniciar	El registro de versión llega como 0x00 o 0xFF: se imprime el mensaje de error de comunicación. Al reconectar y reiniciar vuelve a detectarlo.

UIDs leídos: llavero 99 EB 7B 63 y tarjeta AA 25 60 E4. PENDIENTE: capturas del Monitor Serie y video (ver Terminal/ y Video/).

Preguntas de la práctica

1. ¿Qué significan las siglas SCK, MOSI, MISO y CS?

SCK (*Serial Clock*) es el reloj que genera el Arduino para marcar cada bit. MOSI (*Master Out, Slave In*) lleva los datos del Arduino al lector. MISO (*Master In, Slave Out*) trae los datos del lector al Arduino. CS (*Chip Select*, también llamado SS) activa al dispositivo con el que se quiere hablar: el Arduino lo pone en LOW para seleccionarlo.

2. ¿Por qué el pin del módulo dice "SDA" si no es I2C? ¿Qué función cumple en realidad?

El chip MFRC522 puede comunicarse por SPI, I2C o UART, y el fabricante del módulo rotuló el pin con su nombre de I2C. En esta práctica se usa SPI, así que ese pin funciona como SS/CS: está conectado a D10 y el Arduino lo baja a LOW para indicarle al lector que la comunicación es con él.

3. ¿Por qué línea viaja el UID del lector al Arduino?

Por MISO (D12), que es la única línea que va del lector hacia el Arduino. El Arduino envía las órdenes por MOSI y el lector responde con los bytes del UID por MISO, sincronizados con SCK.

4. ¿Qué pasó al desconectar MISO y por qué?

Al reiniciar, el Monitor Serie mostró "ERROR: no hay comunicacion con el RC522". El programa lee el registro de versión del lector al arrancar; sin MISO la respuesta nunca llega, el Arduino lee la línea al aire (0x00 o 0xFF en lugar de 0x91 o 0x92) y detecta que no hay comunicación. Sin esa línea tampoco puede leer ningún UID, aunque el lector sí reciba las órdenes. Al reconectar el cable y reiniciar, el lector vuelve a detectarse.

5. ¿Por qué no se permitió usar delay() en esta práctica? ¿Qué problema habría causado?

Porque delay() congela todo el programa. Con un delay(2000) para el LED, durante esos 2 s el Arduino no leería tarjetas ni avanzaría el texto de la matriz, y la prueba 3 (acercar la tarjeta no autorizada con el LED encendido) fallaría porque esa lectura se perdería. Con millis() el programa solo revisa si ya pasaron los 2000 ms y sigue atendiendo al lector en cada vuelta del loop().

6. Si quisieran conectar un segundo lector RC522 al mismo Arduino, ¿qué cables compartirían y cuál tendría que ser diferente?

Compartirían el bus: SCK, MOSI y MISO, además de 3.3 V y GND. Lo que tiene que ser diferente es el SS/CS (por ejemplo D8 en lugar de D10), para que el Arduino elija con qué lector habla; nunca se seleccionan los dos a la vez. El RST puede compartirse o ir a otro pin. En el código se crea un segundo objeto, por ejemplo `MFRC522 lector2(8, PIN_RST);`.

Conclusiones

El bus SPI comunica al Arduino con el lector usando líneas compartidas (SCK, MOSI, MISO) y una línea de selección por dispositivo (SS). Leer el registro de versión al arrancar es la forma de saber si de verdad hay comunicación: sin esa revisión, un lector con MISO suelto y un lector sin tarjetas enfrente se verían igual, porque en los dos casos no aparece nada en el Monitor Serie.

El UID sirve como identificador para un control de acceso sencillo, pero se puede copiar con facilidad; un sistema real debería usar también la memoria protegida de la tarjeta o un servidor que valide. La regla de `millis()` de la Práctica 2 volvió a ser la clave: tanto el LED como el texto de la matriz avanzan por tiempo transcurrido, sin detener el programa, y por eso puede negar una tarjeta mientras otra todavía tiene el LED verde encendido.